

Week 2 Study Material

EDA Design Flow Integration: Basic Verilog RTL, Testbenches, and Waveforms

1. Why Week 2 Focuses on Small RTL

Week 1 introduced the complete idea of a design flow: a hardware design begins from a specification and RTL, then produces evidence through simulation, synthesis, implementation, and reports. Week 2 deliberately slows down and focuses on the first technical foundation: reading and simulating very small RTL modules.

This matters because later FPGA and cell-based IC reports are only meaningful when the RTL behavior is already understood. A timing report cannot fix a design whose basic behavior is wrong. A utilization report cannot prove that the output is correct. Therefore, the first evidence habit is functional simulation.

2. Course Scoring Context

Course component	Weight	Why Week 2 matters
Xilinx FPGA evaluation project	45%	Later FPGA implementation reports depend on correct RTL and simulation evidence.
Cell-based IC evaluation project	45%	Later standard-cell synthesis and timing analysis require clean synthesizable RTL.
Attendance and participation	10%	Week 2 lab activity and discussion help build the required evidence habits.

3. Plain Verilog Style Used in This Course

In this course, Week 2 uses beginner-level plain Verilog. The goal is to avoid unnecessary syntax differences while students are still learning how signals, clock edges, reset, and testbenches work.

Use in Week 2	Avoid in Week 2
wire	logic
reg	always_ff
always @(*)	always_comb
always @(posedge clk or negedge rst_n)	SystemVerilog-specific style
.v files	.sv files

4. Synthesizable RTL

Synthesizable RTL is code that an EDA tool can convert into a hardware structure. In Week 2, synthesizable RTL should describe simple combinational logic or simple clocked registers. It should not rely on simulation-only constructs such as time delays inside the hardware module.

RTL construct	Hardware meaning	Beginner note
assign y = a & b;	Combinational logic	Output follows current inputs.
always @(*)	Combinational decision logic	Assign every output in every branch.

RTL construct	Hardware meaning	Beginner note
always @(posedge clk or negedge rst_n)	Clocked register logic	Normal state updates at rising clock edges; active-low asynchronous reset can clear state immediately.
reg q;	Signal assigned inside an always block	May represent stored state in a clocked block.
#10 delay	No synthesizable hardware meaning inside RTL	Use only in testbenches.

5. Verilog Module Structure

A module is a hardware block. It has a name, input/output ports, internal declarations if needed, and logic statements.

```
`timescale 1ns/1ps
module mux2 (
    input  wire sel,
    input  wire a,
    input  wire b,
    output wire y
);

assign y = sel ? b : a;

endmodule
```

- module mux2 names the hardware block.
- input wire declares signals entering the module.
- output wire declares a signal leaving the module.
- assign describes continuous combinational behavior.
- endmodule closes the module.

`timescale 1ns/1ps` tells the simulator the time unit and precision. In this course, keep it at the top of each Verilog source and testbench file for consistent Vivado/XSim simulation.

6. Combinational Logic

Combinational logic has no memory. Its output depends only on the current input values. If the inputs change, the output can change without waiting for a clock edge.

```
assign y = sel ? b : a;
```

The same behavior can also be written with an always @(*) block:

```
reg y;

always @(*) begin
    if (sel)
        y = b;
    else
        y = a;
end
```

end

The most important beginner rule is that every output assigned in a combinational always block must receive a value in every possible path. Otherwise, a synthesis tool may infer a latch.

In plain Verilog, a signal assigned inside an `always` block must be declared as `reg`, even if it does not store state.

7. Sequential Logic

Sequential logic stores values across clock cycles. A register updates when the selected clock edge occurs. Reset places the stored value into a known starting state.

```
`timescale 1ns/1ps
module reg8 (
    input wire    clk,
    input wire    rst_n,
    input wire [7:0] d,
    output reg [7:0] q
);

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 8'd0;
    else
        q <= d;
end

endmodule
```

Here, `d` is the input data signal, and `q` is the stored register output. If `d` changes between two rising clock edges, `q` should not change immediately. `q` updates at the next rising edge, unless reset is active.

8. Blocking and Nonblocking Assignments

For this course, use a simple beginner rule. Use blocking assignment `=` in combinational always blocks. Use nonblocking assignment `<=` in clocked always blocks.

Context	Recommended assignment	Example
Combinational always block	<code>=</code>	<code>always @(*) begin y = a & b; end</code>
Clocked always block	<code><=</code>	<code>always @(posedge clk) begin q <= d; end</code>

This rule avoids many beginner mistakes and matches the way the Week 2 examples are written.

9. What a Testbench Does

A testbench is simulation-only code. It is not synthesized. Its job is to instantiate the design under test, drive input values, generate clock and reset if needed, and check whether outputs match expected behavior.

Testbench part	Purpose
DUT instance	Connects the RTL module to testbench signals.
Clock generator	Creates repeated clock transitions for sequential logic.
Reset sequence	Starts the design in a known state.
Stimulus	Applies input values and operation sequences.
Checker	Compares actual output with expected output.
Log/waveform	Provides evidence for the result.

10. Example Testbench Pattern

```

module tb_mux2;

reg sel;
reg a;
reg b;
wire y;

mux2 dut (
    .sel(sel),
    .a(a),
    .b(b),
    .y(y)
);

initial begin
    sel = 1'b0; a = 1'b0; b = 1'b1;
    #10;
    if (y !== 1'b0) $display("FAIL case 1");
    else          $display("PASS case 1");

    sel = 1'b1;
    #10;
    if (y !== 1'b1) $display("FAIL case 2");
    else          $display("PASS case 2");

    $finish;
end

endmodule

```

The # delays and \$display statements are acceptable here because this is testbench code. They should not be placed inside synthesizable RTL modules for this course.

11. Running Simulation

The exact server setup may vary, but the simulation pattern is usually compile, elaborate, run, and inspect evidence. If the instructor provides a script, use the script so that all students follow the same setup.

```

xvlog -nolog rtl/mux2.v tb/tb_mux2.v
xelab -nolog tb_mux2 -debug typical -s tb_mux2_sim

```

```
xsim -nolog tb_mux2_sim -runall
```

For the sequential register example, the same pattern is used with reg8.v and tb_reg8.v.

12. Reading Waveforms

A waveform shows signal values over time. For combinational logic, check that the output follows the current input values. For sequential logic, check values around clock edges and reset release.

Signal	What to inspect
clk	Find rising edges where registers update.
rst_n	Find when reset is asserted and released.
d or input data	Check what value is available before a clock edge.
q or output data	Check when the stored output changes.
valid/done if present	Check when the result should be considered meaningful.

13. Understanding Unknown Values

In simulation, X means unknown. Some X values before reset may be expected. Persistent X values after reset are usually a warning that something is not initialized, not assigned, or not connected correctly.

- Do not judge normal behavior before reset is released.
- Check that important registers become known after reset.
- Check that the testbench drives all required input signals.
- If X remains, debug one signal at a time.

14. How Week 2 Supports Week 3

Week 3 begins the guided FIR filter project. A FIR filter uses delay registers, arithmetic, and control. The register example in Week 2 is the first step toward understanding the delay line in a FIR filter.

Week 2 concept	Week 3 FIR connection
Multiplexer / combinational selection	Choosing data paths or control-dependent behavior.
Register with reset	One-cycle delay element used in a delay line.
Waveform reading	Checking that input samples move correctly through the design.
Simulation log	Evidence that a small test case passes.
Short explanation	Connecting observed behavior to design intent.

15. Small FIR Concepts to Know Before Week 3

A FIR filter uses a sequence of stored input samples. Each clock cycle, the newest sample enters the circuit, and older samples move through delay registers.

At a high level, a small FIR filter computes:

```
output = coefficient0 × current_sample
        + coefficient1 × previous_sample
        + coefficient2 × older_sample
        + ...
```

For Week 3, you do not need to design the full filter independently. You should only recognize these ideas:

Concept	Why it matters for FIR
Register	Stores one sample from a previous clock cycle.
Delay line	Chain of registers holding recent input samples.
Multiplier	Multiplies a sample by a fixed coefficient.
Adder	Adds partial products.
Reset	Clears stored samples before normal operation.
Testbench	Applies sample values and checks expected outputs.
Waveform	Shows samples moving through registers over time.

16. Self-Check Questions

What is the difference between RTL code and testbench code?

Why should # delays not be used inside synthesizable RTL for this course?

When does a register output update?

Why is reset important in waveform reading?

What evidence should you save after a simulation passes?

How does a Week 2 register relate to a Week 3 FIR delay line?
